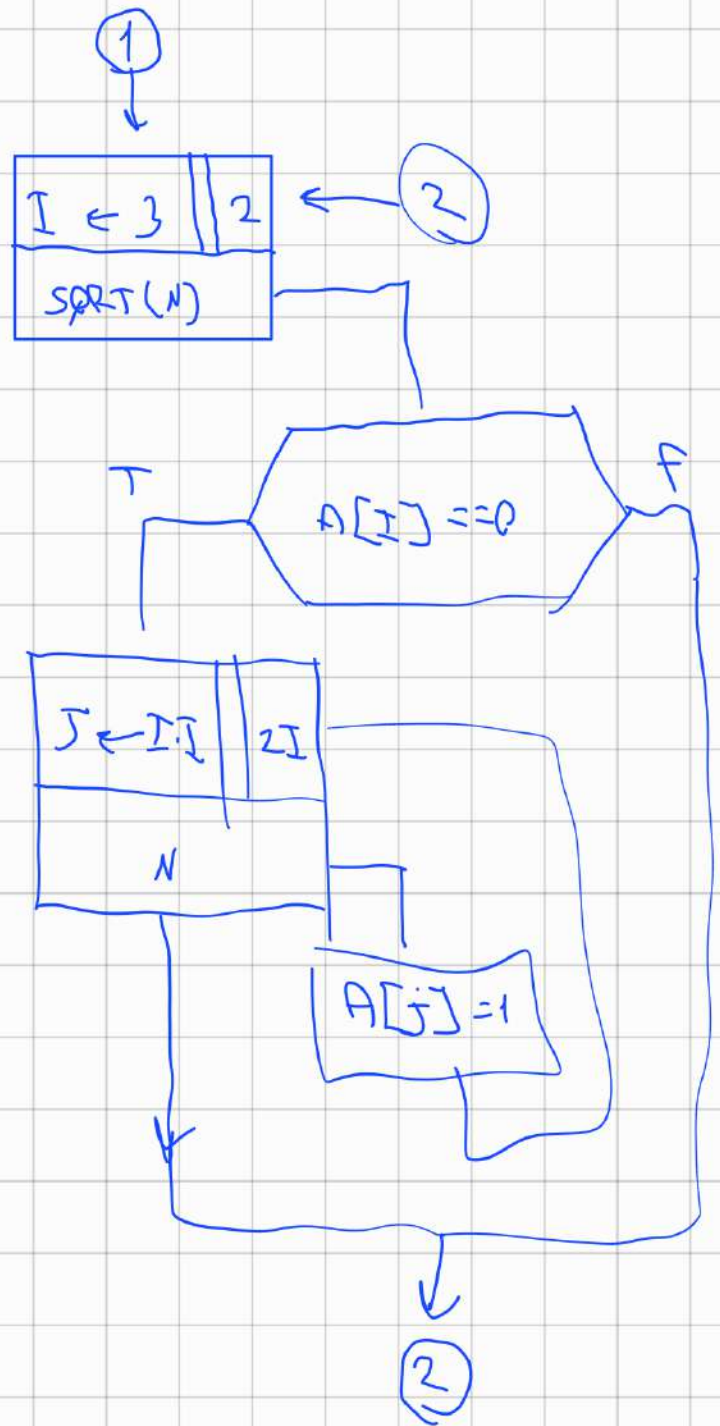
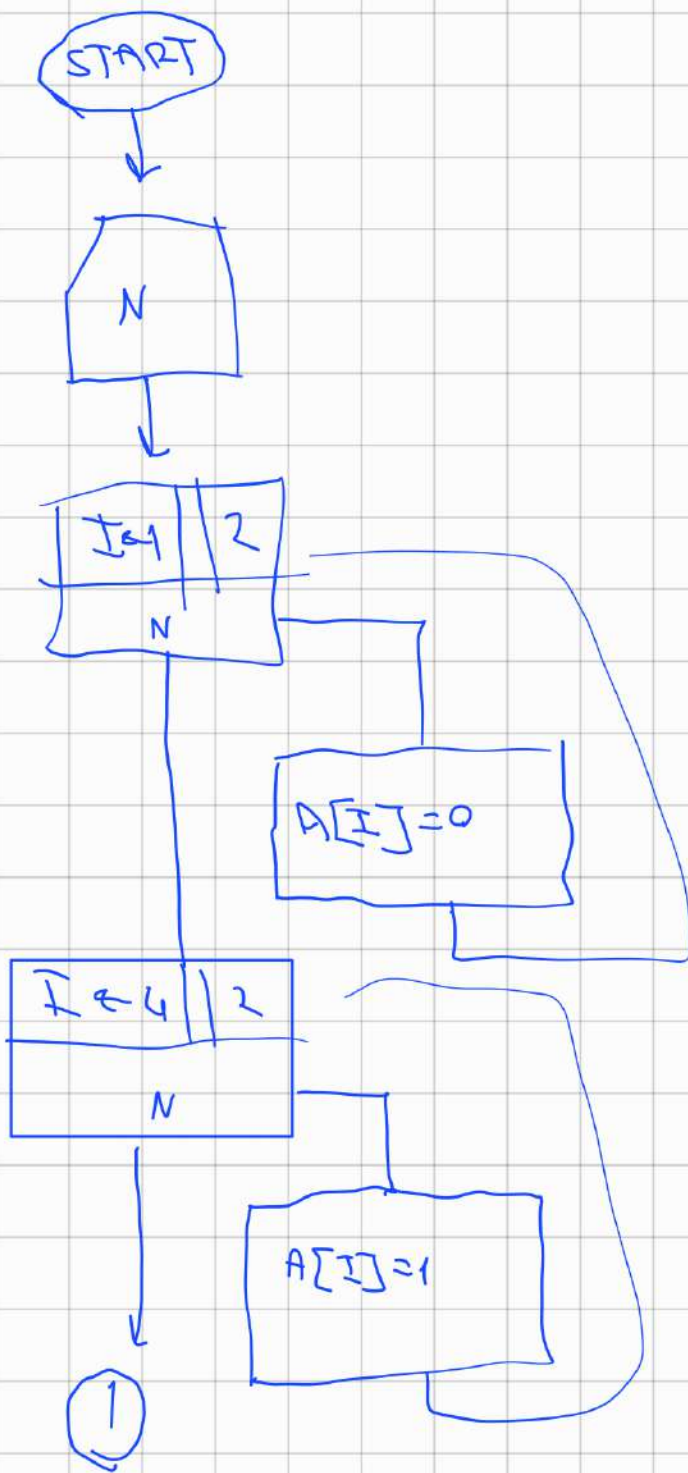


n sayısının asal olup olmaması

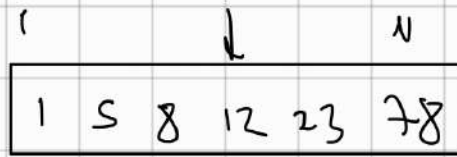


Search Algorithms

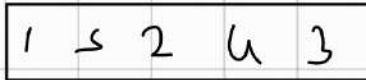
- Linear search
- Binary search
- Binary search tree
- Interpolation search
- Exponential search

- Hashing
- Jump search

- Sorted
- unsorted



Divide and Conquer
Binary Search $O(\log_2(N))$



Linear search $O(N)$

Algorithm Analysis

- Time complexity
- Space complexity

worst case
 Average case
 Best case

Basic operation

- Comparison operation
- Assignment operations (Atoma)
- Arithmetic operations
- Initialization

Example Pseudo code (Finding Maximum Element)

```

currentMax ← A[0]
for i ← 1 to n-1 do
  if A[i] > currentMax then
    currentMax ← A[i]
return currentMax
  
```

Basic operation

2

$2 + n$

$2 \cdot (n-1)$

$2 \cdot (n-1)$

$2 \cdot (n-1) \rightarrow$ increment counter

↑

$$\text{Total } 7n - 1 \quad O(7n - 1)$$

$$\rightarrow O(n)$$

Jump Search

1 5 8 12 28 32 42 53 68 72 73

42 aramışsa $\sqrt{n}$ birim ileri gidilir. Gidilen değerin 42'den

Sürekli ilk elemana elhi dönüp $\sqrt{n}$ elemana kadar Linear search yapar. Küçükse $\sqrt{n}$ elemandan $\sqrt{n}$ birim elemana kadar search yapar.

Interpolation Search / İnterpolasyon

2, 4, 6, 8, 10, 12, ..., 100

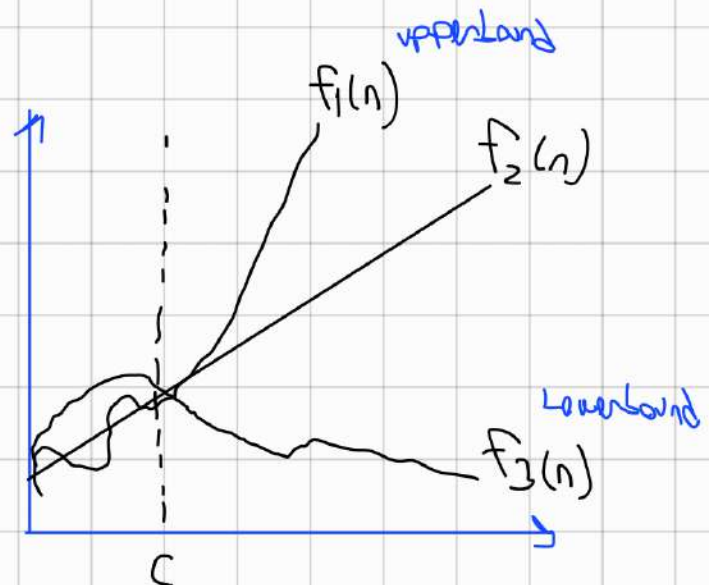
Eğer bir dizi linear olarak artıyor gösteriyorsa bu linear formüle göre search yapılır.

Asymptotic Notation

Big Oh (O) notation: upper bound

Big Theta (Θ) not.: Tight bound

Big Omega (Ω) not.: Lower bound



Functions Growth rate

1 $\rightarrow$ $\log(n)$ n $n \log(n)$ n^2 n^3 2^n $n!$

Linear Search Algorithm

1, 3, 5, 2, 4 $O(1) \rightarrow$ best case
 $O(n) \rightarrow$ worst case

1, 2, 3, 4, 5 $O(1) \rightarrow$ best case
 $O(n) \rightarrow$ worst case

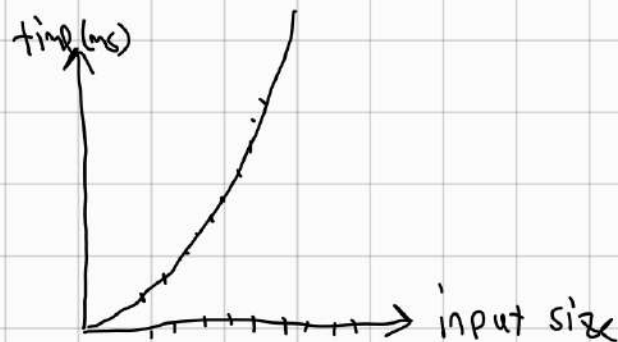
Max-Min element array

unsorted: $O(n) \rightarrow$ best case
 $O(n) \rightarrow$ worst case

sorted: $O(1) \rightarrow$ best case
 $O(1) \rightarrow$ worst case

Time complexity: Number of Inputs (N), Type of Inputs

- Theoretical Analysis
- Experimental Evaluation



Recursion vs Iteration

Criteria

Iteration

Recursion

- mode of implementation

using loops

Function call itself

- State

Defined by the control variable's value

Defined by the parameter values stored in stack

- Progression

The value of control variable moves toward

The function state converges toward the base case

the value in condition

- | | | |
|------------------------|---|--|
| - Termination | Loop ends when control variable's value satisfies the condition | Recursion ends when base case becomes true |
| - Code Size | Iterative code tends to be larger in size | Recursion decreases the size of code |
| - No termination state | Infinite Loops use CPU cycles | Infinite Recursion may cause stack overflow error or it might crash the system |
| - Execution | Faster | Slower |

Factorial Program using Recursion

```
int Factorial (int n) {  
    if (n <= 1) // base case  
        return 1;  
    return n * Factorial (n-1);  
}
```

✓
✗ Recursion

```
int usfAlma (int x, int y) {
```

```
    if (y >= 1)
```

```
        return x * usfAlma (x, y-1);
```

```
return x + istama(x, y-1);
return 1;
}
```

Basamak Bulma Recursion

```
int basamakBul(int n) {
    if (n/10 == 0)
        return 1;
    return 1 + basamakBul(n/10);
}
```

İkinci yöntem

```
- - - - {
static int count = 0; // Fak. birdaha
                        çağrıldığında değeri
                        değişmez.
    f(n) {
        count++;
        basamakBul(n/10);
    }
    return count;
}
```

Fibonacci Numbers

```
int Fibonacci(int n) {
    int a = 0;
    if (n == 1)
        return 1;
    a = 0; b = 1;
    for (i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
}
```

Recursion version

```
int Fibo(int n) {
    if (n <= 1)
        return n;
    return Fibo(n-1) + Fibo(n-2);
}
```

Stack (Abstract Data Type)

First in Last out or Last in First out



Applications: Evaluation of Arithmetic OP.,

$$A + (B * (D / E * F)) \text{ giv}$$

Recursive Func Usage

function call

```
int main() {  
    f1()  
}
```

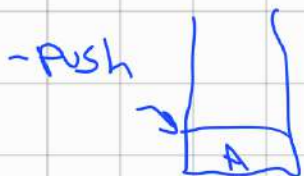
```
void f1() {  
    f2()  
}
```

```
void f2() {  
    f(3)  
}
```



Crossing string, Backtracking (Depth First), Parenthesis matching
conversion of decimal to other number system

Stack operations



- isFull - isEmpty

implementation

Stack $\rightarrow$ Array
 $\rightarrow$ Linked List

Stack used in compilers

) a+b (error
a * (b+c)) error

Stack operations

define MAX 10

```
typedef struct {  
    int item[MAX];  
    int top;  
} STACK;
```

```
void initStack (STACK *s) {  
    s  $\rightarrow$  top = 0;  
}
```

```
int isEmpty (STACK *s) {  
    if (s  $\rightarrow$  top == 0)  
        return 1;  
    else  
        return 0;
```

```
int isFull (stack *s) {  
    if (s  $\rightarrow$  top == MAX)  
        return 1;  
    else  
        return 0;  
}
```

```
int Push (int x, STACK *s) {  
    if (isFull(s))  
        return 0;  
    else {  
        s  $\rightarrow$  item[s  $\rightarrow$  top++] = x;  
  
        return 1;  
    }  
}
```

```
int pop (Stack *s, int *x) {
```

```
    int peek (STACK *s, int *x) {  
        int adr;  
        if (isEmpty(s))
```

```

if (!isEmpty(s))
    return 0;
else {
    *X = s->item[--s->top];
    return 1;
}
}

```

```

if (!isEmpty(s))
    return 0;
else {
    adr = s->top - 1;
    *X = s->item[adr];
    return 1;
}
}

```

Arithmetic Expression Evaluation

$(3+5)*2/3+7 \rightarrow \text{infix}$

Precedence

infix

prefix

postfix

highest $\uparrow$

(

A+B

+AB

AB+

*/

A+B*C

+A*BC

ABC*+

+ -

A+B*C+D

+ +A*BCD

ABC*+D+

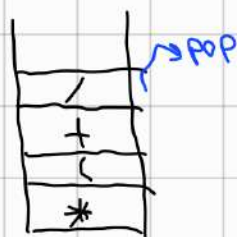
ex)

$a+b*c-d/e*f \rightarrow \text{infix}$

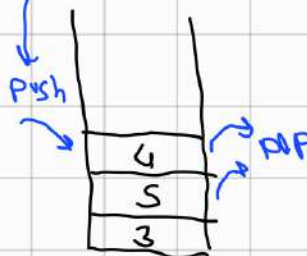
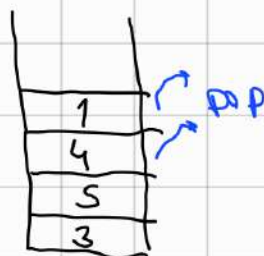
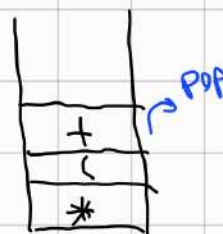
$a+b*c+de/f*- \rightarrow \text{postfix}$

infix convert to postfix

$3*(5+4/1)-2$
 $3\ 5\ 4\ 1\ +\ *\ 2\ -$



$4/1=4$

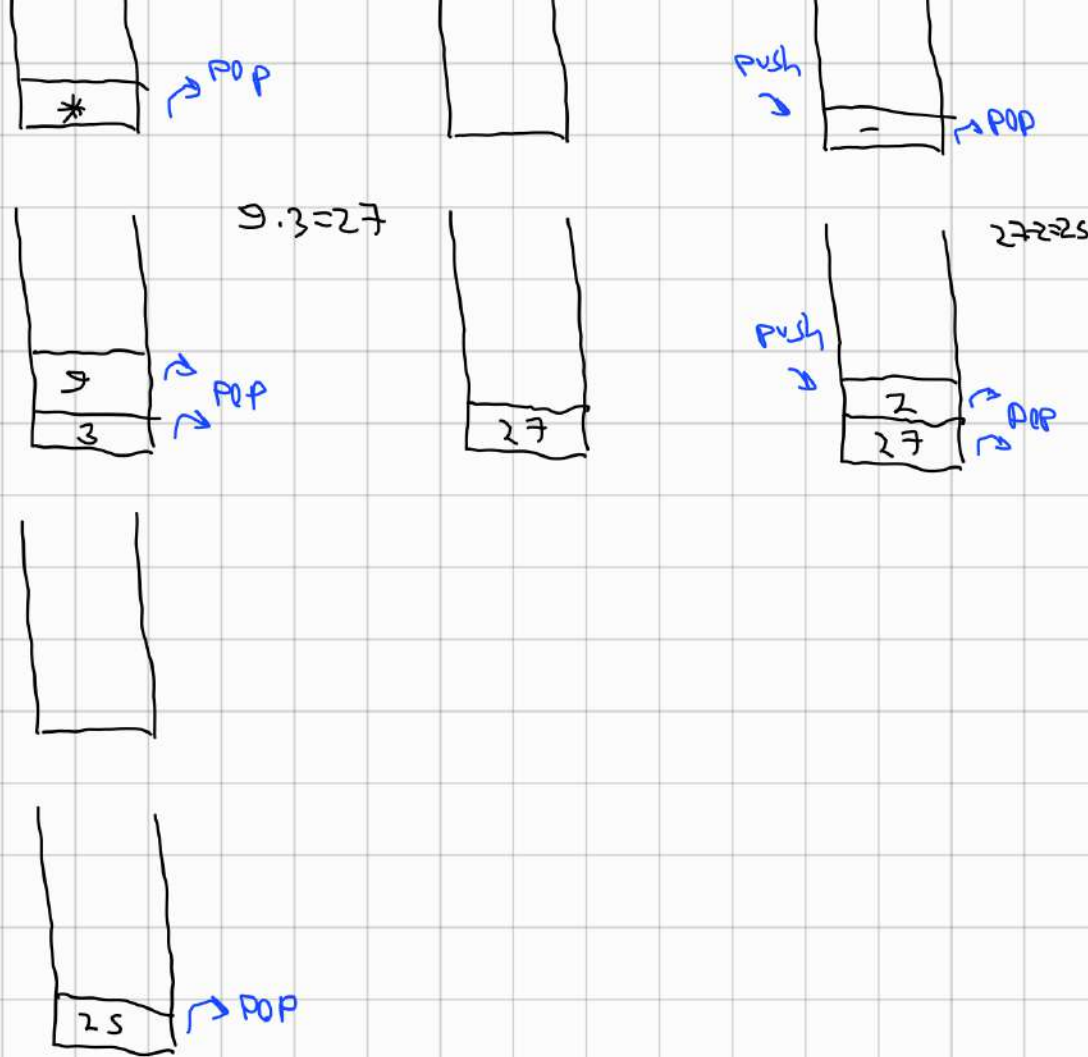


$5+4=9$



push





operator

precedence

Associativity

Exponentiation $^$

Highest

Right Associative

$*$ /

second highest

Left //

$+$ -

Lowest

Left //

Queue (kuzruk)

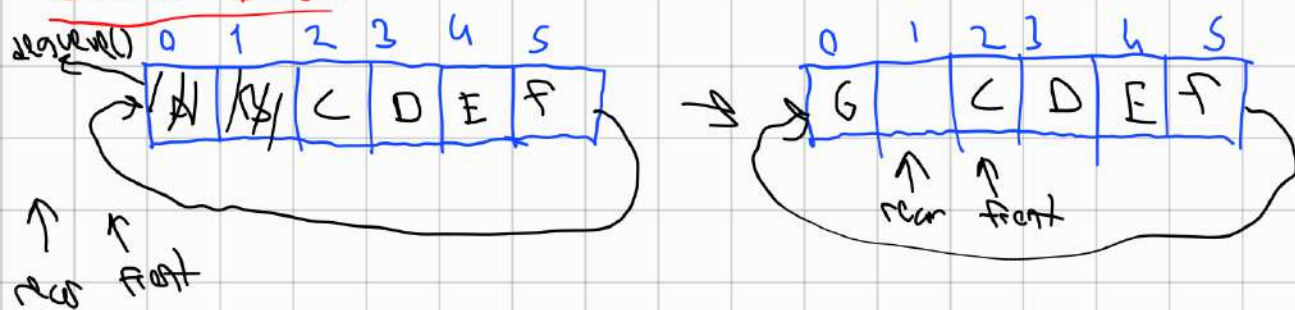
Abstract Data type (səvət veri tipi)



first in first out

- circular queue, Double ended queue, priority queue

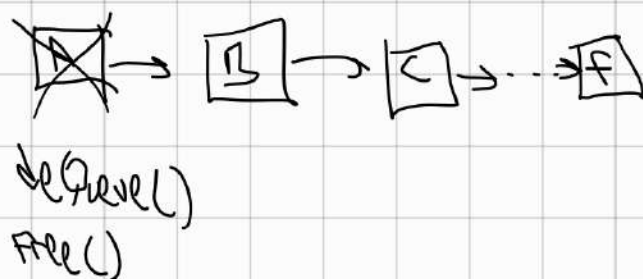
Circular Queue



Queue Operations

- isEmpty()
- isFull()
- enqueue()
- dequeue()
- initQueue()
- peek()

Implementation on Linked List



```
int isEmpty() {  
    if (front == -1)  
        return 1;  
    else  
        return 0;  
}
```

```
int isFull() {  
    if (rear == MAX - 1)  
        return 1;  
    else  
        return 0;  
}
```

```
int dequeue(int *x) {  
    if (isEmpty)  
        return 0;  
    else {  
        *x = q[front++];  
    }  
    return 1;  
}
```

```
int enqueue(int value) {  
    if (isFull()) {  
        return 0;  
    }  
    else {  
        q[++rear] = value;  
    }  
    return 1;  
}
```

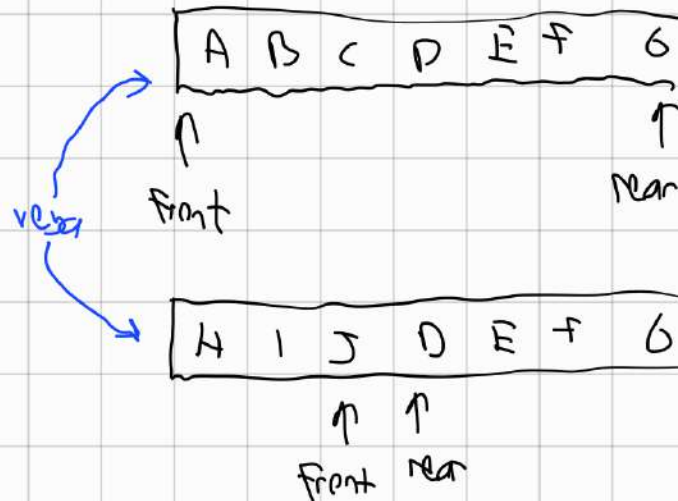
circular Queue optional

```
void initQueue() {  
    int front = -1, rear = -1;  
}
```

```
int isEmpty() {  
    if (front == -1)  
        return 1;  
    return 0;  
}
```

```
int isFull() {  
    if (front == rear ||  
        (front == 0 && rear == SIZE - 1))  
        return 1;  
    else  
        return 0;  
}
```

```
int enqueue(int value) {  
    if (isFull())  
        return 0;  
    else if (front == -1)  
        front = 0;  
    rear = (rear + 1) % SIZE;  
    q[rear] = value;  
    return 1;  
}
```



```
int dequeue(int *x) {  
    if (isEmpty())  
        return 0;  
    else {  
        *x = q[front];  
        if (front == rear) {  
            front = -1;  
            rear = -1;  
        }  
        else  
            front = (front + 1) % SIZE;  
    }
```

return 1;

}

}

Priority Queue

High 

Medium 

Low 

Double Ended Queue



- Input Restricted DeQueue (Double ended Queue)

Input is restricted at a single end but allows deletion at both the ends

- Output Restricted DeQueue

output is restricted at a single end but allows insertion at both the ends

Comparison Stack versus Queue

Stack

LIFO

working principle

structure

same end
used to insert
and delete

Queue

FIFO

one end is used for insert another
end is used for deletion

operations

push, pop

Enqueue, Dequeue

variants

it doesn't have

circular, priority, Double ended Queue

Implementation

Simpler

Comparatively complex

Linear Data structure

- Array
- Linked List
- Stack
- Queue

NON-Linear

(hierarchical Data structure)

- Graphs
- Tree

implementation

Simpler

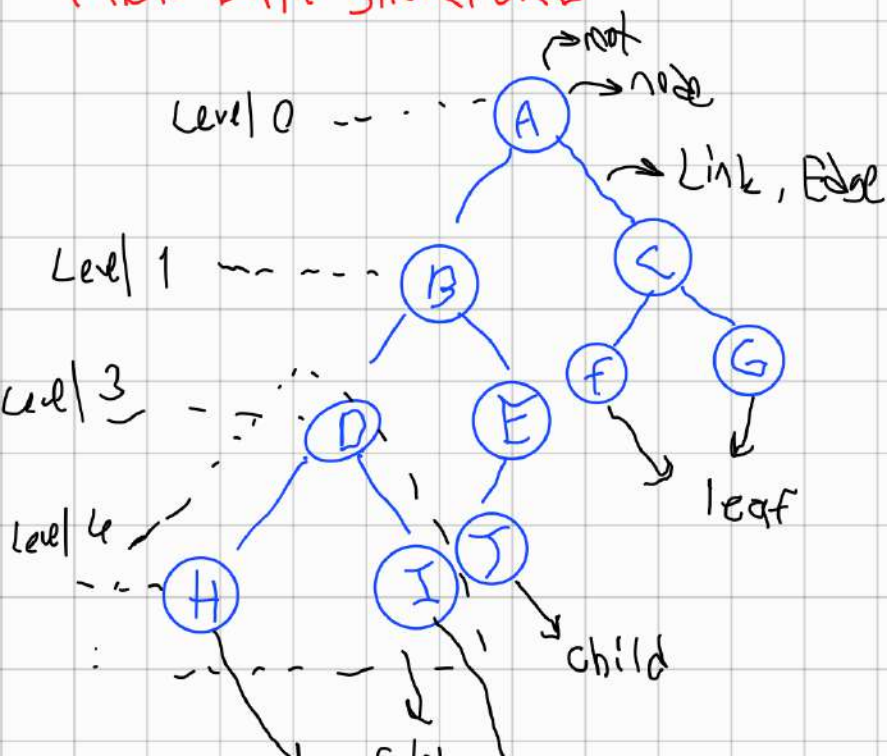
Complex

Levels involved

Single level

Multiple Level

TREE DATA STRUCTURE



Terminology

- Path
- Root
- parent
- child
- Leaf
- subtree
- Visiting
- Level
- Forest

Subtree
Sibling node

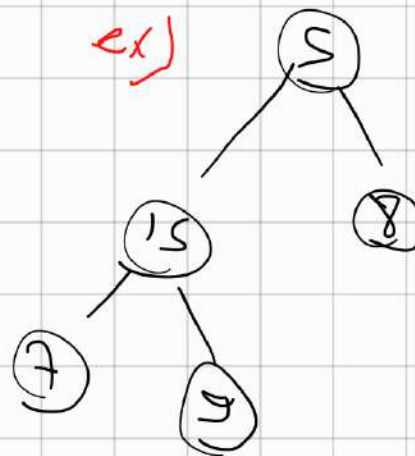
- Depth
- Height

TREE APPLICATIONS

- File Management Systems
- Compression Algorithms (Huffman Algorithm)
- Databases
- Compilers (Syntax Tree)
- Priority Queue
- AI (Decision Tree, Random Forest)
- Indexing multi-dimensional information

Types of Tree

- Binary Tree
- Binary Search Tree
- AVL Tree
- B-Tree
- R-Tree
- Interval Tree
- Heap Tree



1) $5 \rightarrow 15 \rightarrow 8 \rightarrow 7 \rightarrow 9$
from top left to right

2) $7 \rightarrow 9 \rightarrow 15 \rightarrow 8 \rightarrow 5$
from bottom left to right

node traversal

- Inorder traversal $\rightarrow$ left-subtree, node, right-subtree
- Preorder traversal $\rightarrow$ node, left-subtree, right-subtree
- Postorder traversal $\rightarrow$ left, right, node

Inorder traversal

7 $\rightarrow$ 15 $\rightarrow$ 9 $\rightarrow$ 5 $\rightarrow$ 8

preorder traversal

5 $\rightarrow$ 15 $\rightarrow$ 7 $\rightarrow$ 9 $\rightarrow$ 8

Postorder traversal

7 $\rightarrow$ 9 $\rightarrow$ 15 $\rightarrow$ 8 $\rightarrow$ 5

Codes

```
struct node {  
    int data;  
    struct node * left;  
    struct node * right;  
}
```

```
void inorderTraversal(struct node *root) {  
    if (root == NULL)  
        return;  
    inorderTraversal(root  $\rightarrow$  left);  
    printf("%d ", root  $\rightarrow$  data);  
    inorderTraversal(root  $\rightarrow$  right);  
}
```

```
void preOrderTraversal(struct node *root) {
```

```
    if (root == NULL) return;
```

```
    printf("%d ", root  $\rightarrow$  data);
```

```
    preOrderTraversal(root  $\rightarrow$  left)
```

```
    preOrderTraversal(root  $\rightarrow$  right)
```

```

}

void postOrderTraversal(struct node *root) {

```

```

    if (root == NULL) return;

```

```

    postOrderTraversal(root->left);

```

```

    postOrderTraversal(root->right);

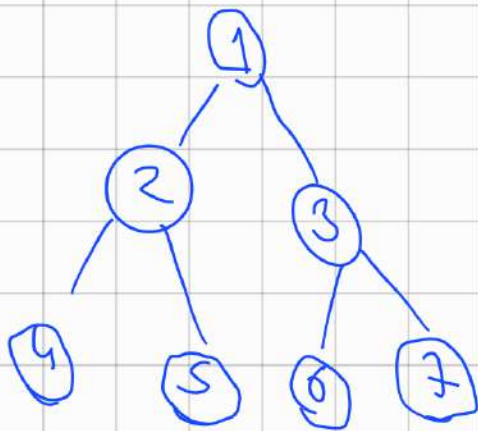
```

```

    printf("%d", root->data);
}

```

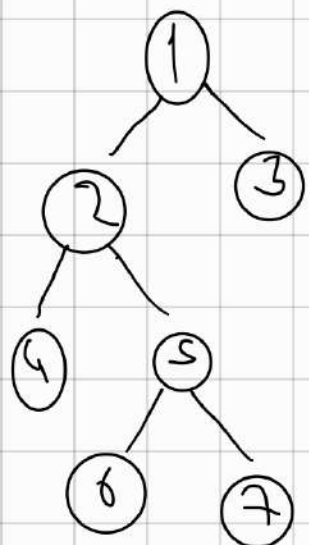
Perfect Binary Tree



it's type of binary tree which every internal nodes has exactly two child nodes and all the leaf nodes are at the same level.

not: eger sola yasliysa complete binary tree

Full Binary Tree

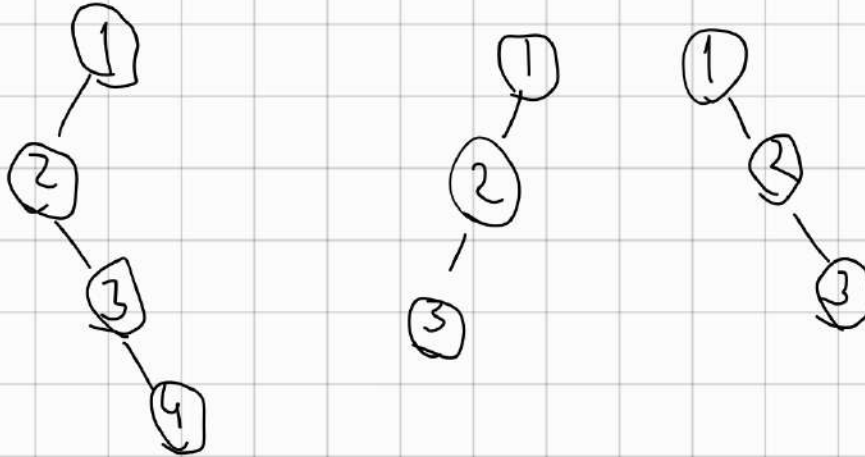


2 veya 0 çocuğu varsa.

Balanced Binary Tree

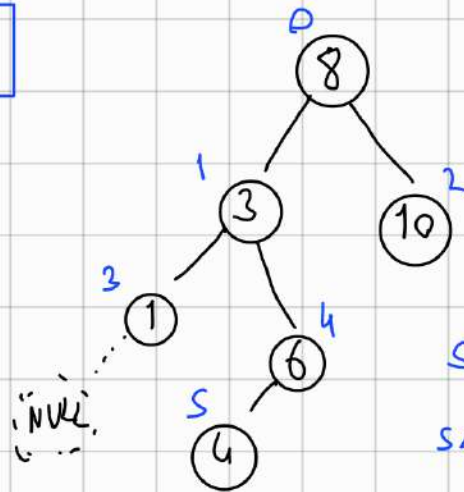
Sol ve sag dallarin yu'kselik farki en fazla 1 olmal

Degenerate OR Pathological Tree



Binary Search Tree

8 3 10 1 6 4



başlangıç

0. index

$2k+1$

1. index

$2k$

Sol

Sag

$2k+2$

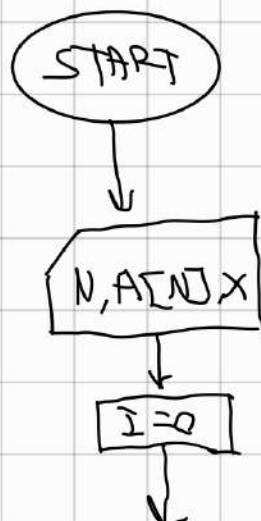
$2k+1$

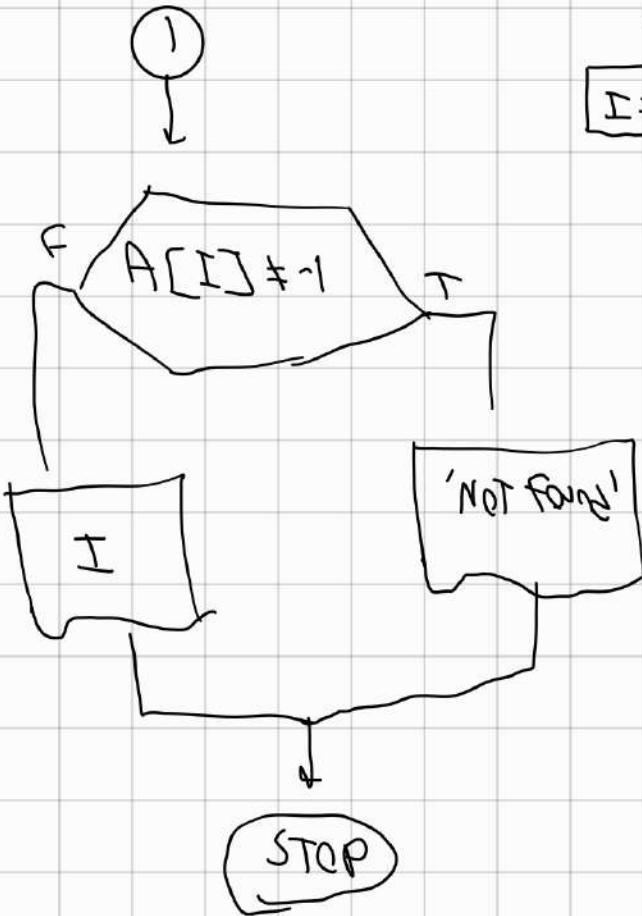
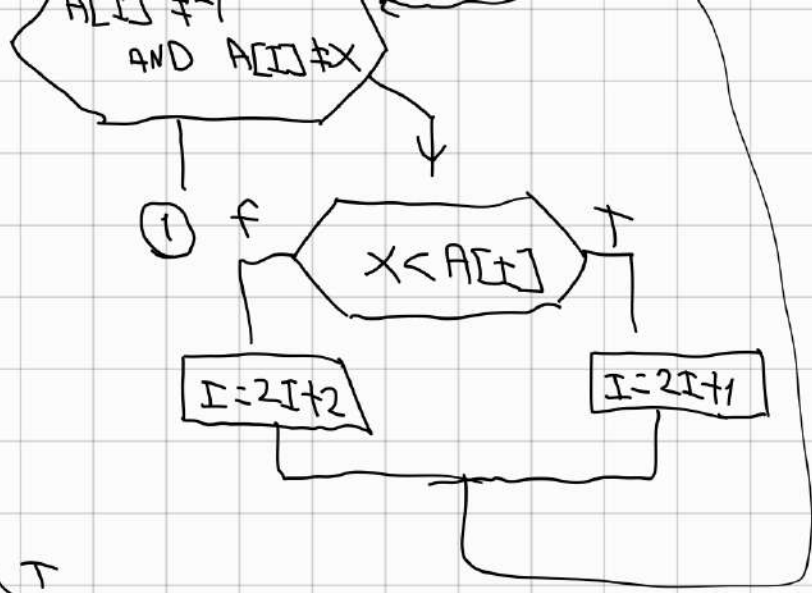
- Search

- Insert

- Delete

Search

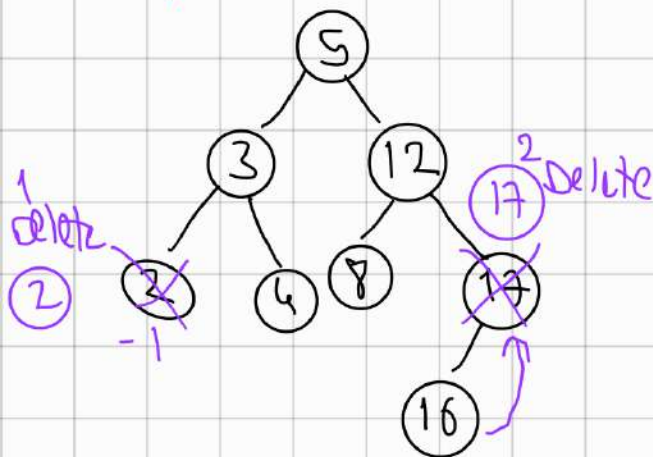




not: Sıralı dizide diziyi 2'ye bölüp ortası almak mantıklı gözündür. Sol orta, Sağ orta sonra Sol

not: Insert ile Search birlikte kalmaz

Delete



Case 1: Delete Leaf node

Case 2: Delete Internal node with a single child

Case 3: Delete Internal node who have right-left child

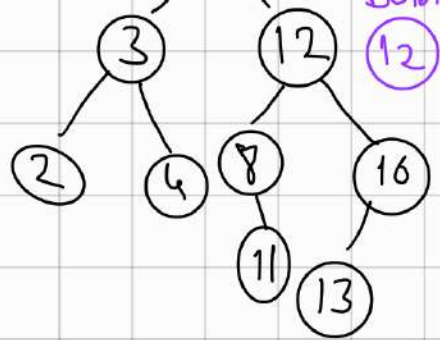
Case 4: Not Found

Case 3:



1-) Get the inorder successor of that node

2-) Replace the node with the inorder successor



3) Remove the inorder successor from its original position

Inorder Successor: a node is the next node in order Traversal of the Binary Tree.

Inorder Successor of an input node can also be defined as the node with the smallest key greater than the key of the input node.

1. If the right subtree of a node is not NULL, then Successor lies in the right subtree.
2. If " " " " " " is NULL, start from root, travel down the tree, if a node's data is greater than root's data then go right side, otherwise go left side.

not: Silinecek elemanın sağında eleman varsa sağın en küçük elemanı alınır, silinecek eleman konur. Eğer sağda eleman yoksa 2.yi oku.

Code

```
struct node *deleteNode (struct node *root, int key){
```

```
    if (root == NULL) return root;
```

```
    if (key < root->key)
```

```
        root->left = deleteNode (root->left, key);
```

```
    if (key > root->key)
```

```
        root->right = deleteNode (root->right, key);
```

```
    else {
```

```
        if (root->left == NULL) {
```

```
            struct node *tmp = root->right;
```

```

free(root);
return tmp;
} else if (root → right == NULL) {
    struct node * tmp = root → left;
    free(root);
    return tmp;
}
struct node * tmp = minValueNode (root → right);
root → key = tmp → key;
root → right = deleteNode (root → right, tmp → key);
}
return root;
}

```

```

struct node * minValueNode (struct node * node) {

```

```

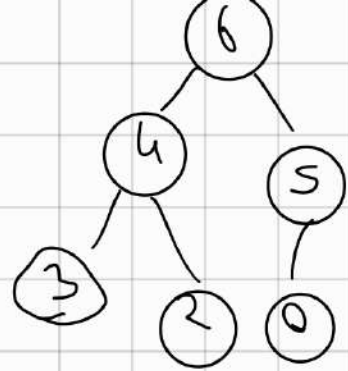
    struct node * current = node;
    while (current && current → left != NULL)
        current = current → left;
    return current;
}

```

Heap Tree

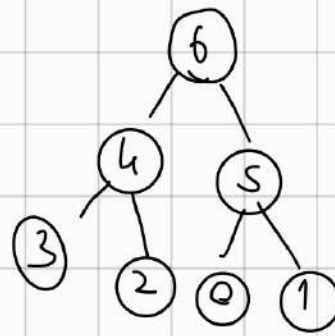
- complete Binary tree

- Max-Heap $A[\text{Parent}(i)] \geq A[i]$
- Min-Heap $A[\text{Parent}(i)] \leq A[i]$



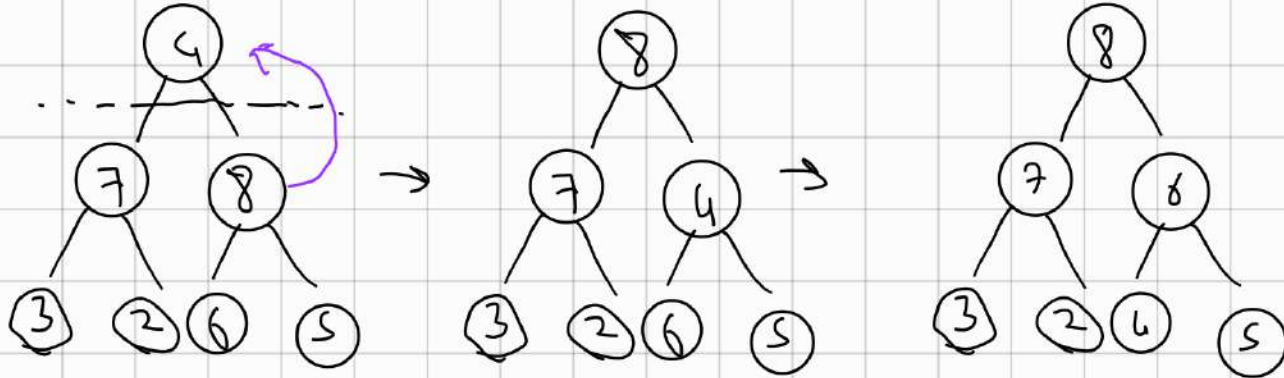
Array:

6	4	5	3	2	0	1
---	---	---	---	---	---	---



Aynı zamanda Complete Binary Tree

Heapfy



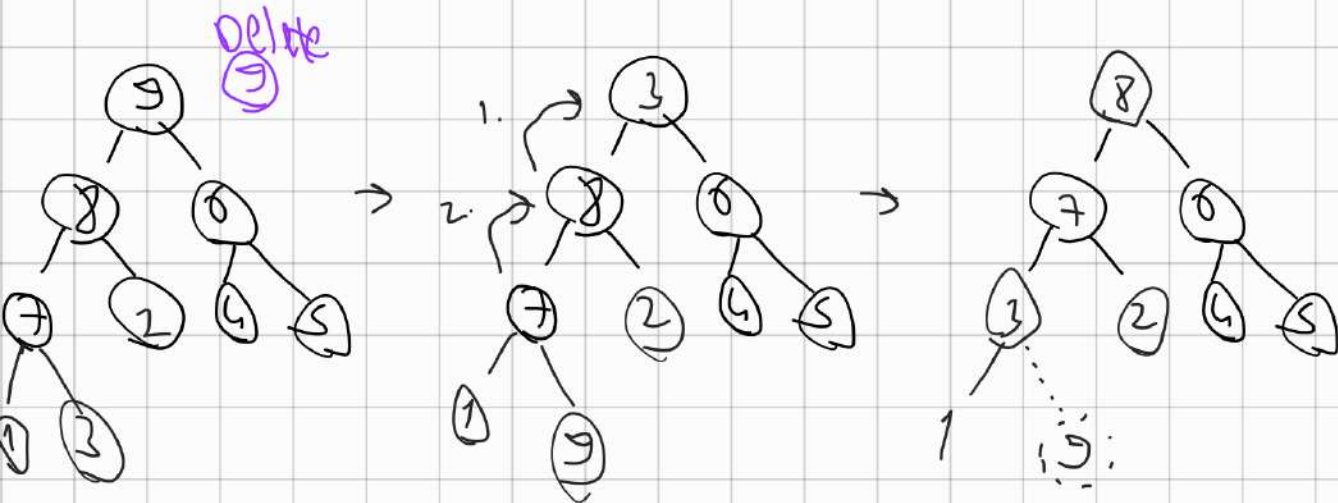
- Insert $\rightarrow$ complexity $\log(n)$

- Delete

- search

Delete operation

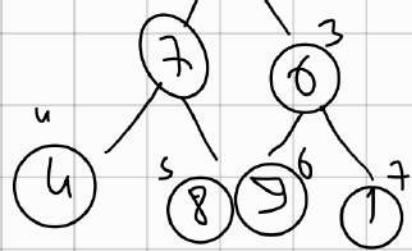
Sililmek elemanı en sona atarız. Sonu sileriz ve tekrar heapfy yaparız.



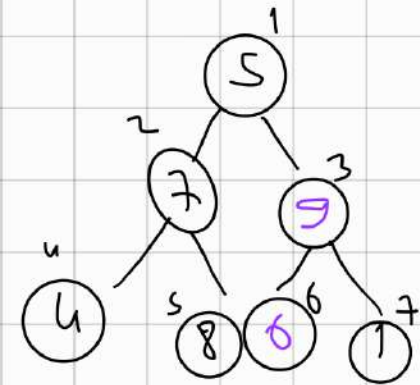
Max Heap-Tree Conversion



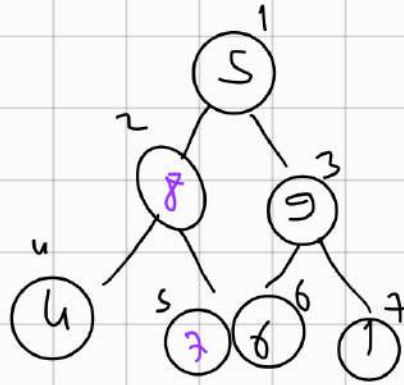
$$N/2 \quad 7/2 = 3$$



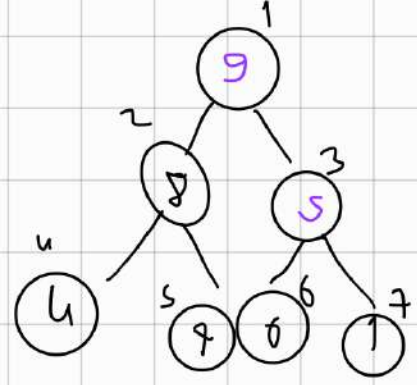
STEP 1



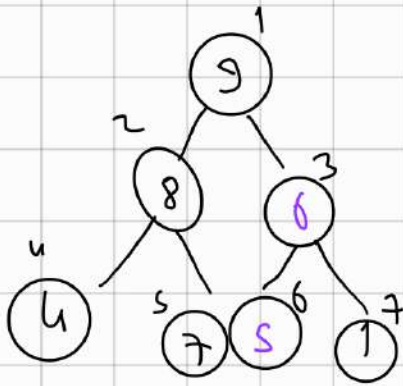
STEP 2



STEP 3



STEP 4



Code

```
void buildMaxHeap(int dizi[]) {
```

```
    int i;
```

```
    for (i = N/2; i > 1; i--)
```

```
        maxHeapify(dizi, i, N);
```

```
}
```

```
void maxHeapify(int dizi[], int i, int N) {
```

```
    int largest;
```

```
    int left = 2 * i;
```

```
    int right = 2 * (i + 1);
```

~~if((left < N) && dizi[left] > dizi[i])~~

largest = left;

else

largest = i;

if((right < N) && dizi[right] > dizi[largest])

largest = right;

if(largest != i) {

swap(dizi[i], dizi[largest]);

maxHeapify(dizi, largest, N);

}

}

Heap Sort

Heapify edilmiş ağacın en büyük elemanı en son elemanı ve tekrar heapify edilir.
Daha sonra N bir azaltılır. Döngü devam eder.

